Software Requirements Specification (SRS)

Document Control

- Product Name: `Root` Association Management App
- Version: `1.1`
- Date: `2026-04-19`
- Prepared For: `Root` Association
- Status: Draft for Review
- Change log:
  - `1.1`: Member submission requests with authorization before ledger update; Super Admin and permission delegation; Finance/Accounts investment and fund-release workflow; payment evidence (channel, trx ID, notes, optional image); roadmap org roles documented.

1. Introduction

1.1 Purpose

This SRS defines functional and non-functional requirements for the `Root` association app. The system manages members, capital ledger, investments, and snapshot-based profit/loss distribution so financial allocations remain historically correct.

1.2 Scope

Version 1 (MVP) includes:

- Super Admin, Admin, and Member (User) authentication.
- Administration: Super Admin assigns **permissions** (who may approve submissions, create investments, release funds, distribute, etc.).
- Member profile management.
- **Member-initiated capital submissions (e.g. installments)** as **pending requests**; authorized staff approve or reject before any member balance or ledger update.
- Payment **evidence** on each request: channel (e.g. hand cash, bKash, bank), optional transaction/reference ID, free-text notes, optional **image** proof (slip/screenshot).
- Capital ledger transactions: member flow via approved requests; **admin-direct** postings for `SUBMISSION`, `WITHDRAW`, `ADJUSTMENT` where policy allows.
- Investment lifecycle: creation by Finance (permissioned role), **fund release** by Accounts (permissioned role), snapshot at defined lifecycle point, close, distribute.
- Snapshot-based profit/loss distribution.
- Audit logs and member statements (including pending vs posted submissions).
- Reports and exports.

- Approval queue for authorized roles.

Version 1 excludes:

- **Live** external banking or mobile-wallet API integration (verification is **manual** using trx ID, notes, and attachments).
- Automatic market data ingestion.
- Tax filing automation.

1.3 Definitions

- Member: Association participant with capital balance.
- Capital: Member net contributed amount derived from **authorized** ledger entries only.
- **Submission request**: Member-created proposal to increase capital (installment or general submission) that remains **pending** until authorized.
- **Authorization**: Action by a permissioned user to approve or reject a submission request; approval creates the corresponding ledger entry.
- **Super Admin**: User who manages other users and **grants or revokes capabilities** (permissions), not only a fixed role menu.
- **Payment channel**: How money moved (e.g. `HAND_CASH`, `BKASH`, `BANK`, `OTHER`) for documentation and workflow (e.g. hand cash may stay pending until Accounts confirms physical receipt).
- **Pending receipt**: A submission request that is not yet approved; it does **not** change capital or investment snapshot totals.
- Snapshot: Frozen ratio of each member's capital to total capital at investment creation time (capital counts **authorized** balances only; pending requests are excluded).
- Distribution: Posting of investment profit/loss shares to member ledgers.
- Investment P/L: `returnAmount - investedAmount`.
- Ledger Entry: Immutable financial record affecting a member balance.

1.4 Stakeholders

- Association Board / Owners
- Admin Operators
- Members
- Auditor / Accountant
- System Administrator

2. Product Overview

2.1 Product Perspective

The application is a financial ledger and investment allocation platform tailored for `Root`. Core trust mechanism is historical snapshot preservation.

2.2 User Classes (MVP)

Capabilities below are **permission-gated** unless noted; Super Admin assigns which user has which permission.

- **Super Admin**:
  - Manage users (create/update/deactivate Admin and Member accounts).
  - Assign and revoke **permissions** (see § 3 FR-01) for who may approve submissions, create investments, release funds, close investments, distribute, reverse, post admin-direct ledger entries, and run reports.
- **Admin** (operational; exact powers depend on granted permissions):
  - Typical bundles: approve/reject submission requests; admin-direct ledger postings; investment and distribution operations; reports — as granted by Super Admin.
- **Member (User)**:
  - Sign in.
  - Create **submission requests** (installments / capital submissions) with payment evidence; view status `PENDING`, `APPROVED`, `REJECTED`.
  - View own profile, **effective** balance (from authorized ledger only), statement, investment-wise distribution history, and list of pending requests.
  - Cannot change balance without an authorized approval flow or admin-direct policy.

2.3 Operating Environment

- Primary stack target: Flutter client + secure backend API + relational database.
- Final deployment target (Android/iOS/Web) remains a business decision. SRS requirements are platform-neutral.

2.4 Future organizational roles (roadmap)

The following titles are **out of MVP enforcement** as separate system roles; they may later map to **permission bundles** or named roles: CEO, MD, Accountant, Finance, Treasurer. MVP implements **Super Admin**, **Admin**, and **Member** plus fine-grained **permissions** so future titles do not require a schema break.

3. Functional Requirements

FR-01 Authentication, Session, and Authorization

- System shall support secure login for Super Admin, Admin, and Member.
- System shall enforce **permission-based** access control for all protected features (not solely a coarse role enum).
- Super Admin shall manage **permission grants** per user (e.g. `APPROVE_SUBMISSION`, `CREATE_INVESTMENT`, `RELEASE_INVESTMENT_FUNDS`, `CLOSE_INVESTMENT`, `DISTRIBUTE_PL`, `REVERSE_DISTRIBUTION`, `POST_ADMIN_LEDGER`, `VIEW_ALL_REPORTS`).

- System shall provide logout and session expiration.

FR-02 Member Management

- Super Admin (and permissioned Admin, if policy allows) shall create, update, and deactivate member profiles.
- Member profile fields include name, contact number, email, join date, status, and optional notes.
- Deactivated members remain visible in historical reports and calculations.

FR-03 Member submission requests (installments / capital submissions)

- Member shall create a submission request with: amount, request date, `request_type` (e.g. `INSTALLMENT`, `SUBMISSION`), **payment channel**, optional **external reference** (trx ID / bank reference), **notes** (e.g. handoff to another person to deposit), and optional **attachment** (image or PDF proof).
- Request status shall be `PENDING` until approved or rejected.
- Pending requests shall **not** affect member balance, capital totals, or investment snapshots.

FR-04 Authorization of submission requests

- Users granted `APPROVE_SUBMISSION` (or equivalent) shall review the **approval queue**.
- Authorizer shall **approve** or **reject** with optional `rejection_reason`.
- On **approve**, system shall atomically create exactly one `SUBMISSION` (or policy-mapped) **ledger entry** for the member, link it to the request, set request status to `APPROVED`, and record authorizer and timestamp.
- On **reject**, no ledger entry shall be created; status `REJECTED`; reason stored.
- Business meaning of **pending** may depend on channel (e.g. bank/bKash: verify reference; hand cash: confirm receipt by Accounts); SRS does not mandate auto-verification — manual check against supplied evidence.

FR-05 Capital ledger management (admin-direct)

- Users granted `POST_ADMIN_LEDGER` shall post these transaction types when policy allows: `SUBMISSION`, `WITHDRAW`, `ADJUSTMENT` **without** a member request (e.g. corrections, operational deposits).
- Member-initiated balance increases for installments go through FR-03/FR-04 unless explicitly configured otherwise.
- **Withdrawals** in MVP follow **admin-direct** posting (no member self-service withdrawal request required unless added in a later version).
- Each transaction shall include date, amount, comment, actor, and timestamp.
- System shall update member current balance after each valid transaction.
- System shall reject postings that would make member balance negative unless explicitly allowed by policy.

FR-06 Investment creation, fund release, and snapshot

- Users granted `CREATE_INVESTMENT` (business role: **Finance**) shall create an investment record in status `DRAFT` with: title/type, invested_to, invested_amount, date, comment.
- Users granted `RELEASE_INVESTMENT_FUNDS` (business role: **Accounts**) shall confirm **fund release** (funds actually available for the investment). System shall set `fund_released_at`, `fund_released_by`, then transition `DRAFT` → `OPEN`.
- At transition to `OPEN`, system shall capture snapshot ratios for all included members using **authorized** capital only (pending submission requests excluded).
- Snapshot data shall be immutable and versioned by investment id.
- Closing and distribution remain permissioned per FR-07, FR-08, and FR-09.

FR-07 Investment closure

- Users granted `CLOSE_INVESTMENT` shall close an open investment by entering: return_amount, closure_date, closure_comment.
- System shall compute profit/loss using snapshot-linked investment data.
- Closed investment cannot be edited except through controlled reversal/correction flow.

FR-08 Profit/Loss distribution

- Distribution trigger is manual per closed investment; requires `DISTRIBUTE_PL` permission.
- System shall calculate member share using stored snapshot ratios.
- System shall support positive and negative distributions.
- On loss, member balances shall be reduced proportionally using the same snapshot ratio.
- Distribution posting shall create immutable ledger entries per member.

FR-09 Idempotency, Reversal, and Correction

- System shall prevent duplicate distribution for the same investment.
- Users granted `REVERSE_DISTRIBUTION` may reverse a distribution through explicit reversal action.
- Reversal shall create new compensating entries; previous entries remain unchanged.
- After successful reversal, investment may be re-distributed once.

FR-10 Statements and reports

- Member shall view:
  - current balance (authorized ledger only)
  - submission requests and their statuses
  - personal transaction history
  - distribution history by investment
- Users with report permissions shall view:
  - association summary

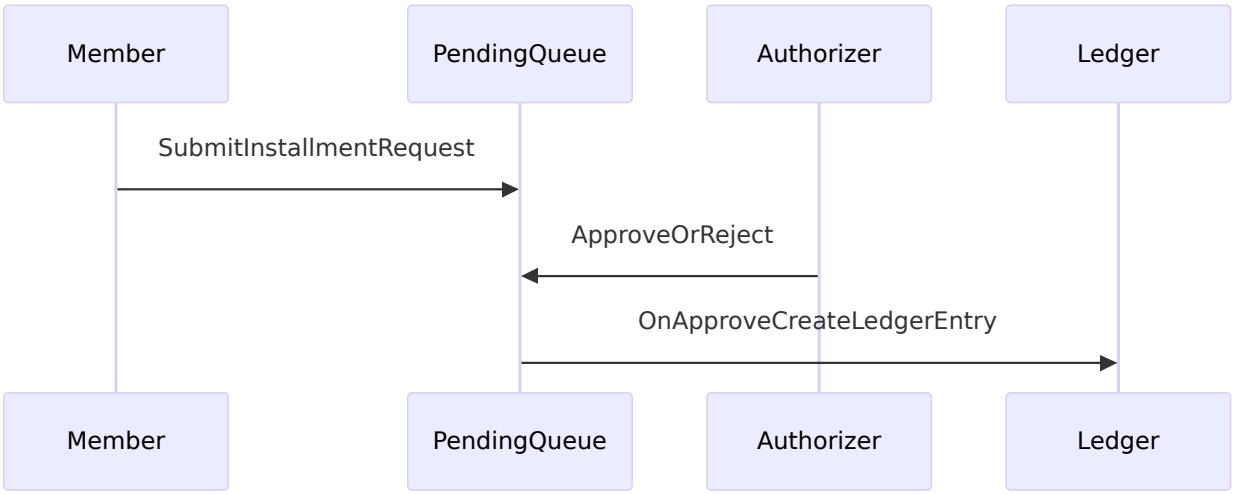
- all member balances
- investment register
- distribution logs
- **approval queue** (pending submission requests)

- System shall provide CSV export for reports.

FR-11 Audit and traceability

- System shall log create/update/approve/reject/close/distribute/reverse actions with actor and timestamp.
- Sensitive operations shall store before and after values.
- Audit logs shall be immutable and queryable by date, actor, and entity id.

3.1 Submission and authorization flow (reference)



4. Financial Calculation Specification

4.0 Capital basis for snapshots and pending requests

- Member **current capital** for snapshot purposes equals sum of **posted** ledger entries (authorized), excluding any `PENDING` submission request.
- **Pending** submission requests shall not appear in `userCapitalAtSnapshot` until approved and posted to the ledger.

4.1 Snapshot Formula

For each investment event that transitions to `OPEN` (or equivalent) with snapshot capture:

- `totalCapitalAtSnapshot = sum(userCapitalAtSnapshot for all included members)`
- `userCapitalAtSnapshot` shall reflect **authorized** capital only.
- `snapshotRatio(member) = userCapitalAtSnapshot / totalCapitalAtSnapshot`

Constraint:

- `sum(snapshotRatio(member)) = 1.0` before rounding storage.

4.2 Investment Profit/Loss

- `investmentPnL = returnAmount - investedAmount`

Interpretation:

- `investmentPnL > 0`: profit distribution.
- `investmentPnL = 0`: no-op distribution allowed for closure consistency.
- `investmentPnL < 0`: loss distribution.

4.3 Member Share Formula

- `rawShare(member) = snapshotRatio(member) * investmentPnL`

4.4 Rounding and Reconciliation Policy

- Currency precision: 2 decimal places.
- Round each member share using half-up to 2 decimals.
- Compute:
  - `roundedTotal = sum(roundedShare(member))`
  - `remainder = investmentPnL - roundedTotal`
- If `remainder != 0`, apply remainder to the member with highest absolute capital in snapshot.
- If tie, apply to lowest `member_id` for deterministic behavior.

4.5 Worked Example (Profit)

Input:

- Snapshot capitals: A=11000, B=6000, C=5000
- Total = 22000
- PnL = +500

Raw shares:

- A: 11000/22000*500 = 250.00
- B: 6000/22000*500 = 136.36
- C: 5000/22000*500 = 113.64

Rounded total = 500.00, remainder = 0.00.

4.6 Worked Example (Loss)

Input:

- Same snapshot
- PnL = -350

Raw shares:

- A: -175.00
- B: -95.45
- C: -79.55

System posts negative ledger entries exactly as rounded shares, reducing balances.

4.7 Distribution Safety Rules

- Distribution requires `investment.status = CLOSED`.
- Distribution allowed only when no active distributed record exists.
- Reversal required before any re-distribution.

5. Data Requirements and Contracts

5.1 Entity: User

Required fields:

- `user_id` (uuid, PK)
- `full_name` (string, non-empty)
- `contact_no` (string)
- `email` (string, nullable, unique if present)
- `join_date` (date)
- `role` (enum: `SUPER_ADMIN`, `ADMIN`, `MEMBER`)
- `status` (enum: ACTIVE, INACTIVE)
- `created_at`, `updated_at`

Notes:

- Fine-grained access is expressed via **UserPermission** (§ 5.3). `SUPER_ADMIN` may bypass checks for administration features per policy.

5.2 Entity: Permission (catalog)

Stable codes for grants (examples; extend as needed):

- `APPROVE_SUBMISSION`, `POST_ADMIN_LEDGER`, `CREATE_INVESTMENT`, `RELEASE_INVESTMENT_FUNDS`, `CLOSE_INVESTMENT`, `DISTRIBUTE_PL`, `REVERSE_DISTRIBUTION`, `VIEW_ALL_REPORTS`, `MANAGE_USERS`

Required fields:

- `permission_id` (uuid or string code, PK)
- `code` (string, unique)

- `description` (string)

5.3 Entity: UserPermission

Required fields:

- `user_permission_id` (uuid, PK)
- `user_id` (FK -> User)
- `permission_id` (FK -> Permission)
- `granted_by` (FK -> User, nullable for bootstrap)
- `granted_at` (timestamp)

Unique:

- (`user_id`, `permission_id`)

5.4 Entity: CapitalSubmissionRequest

Required fields:

- `request_id` (uuid, PK)
- `user_id` (FK -> User; submitting member)
- `request_type` (enum: `INSTALLMENT`, `SUBMISSION`, extensible)
- `amount` (decimal(18,2), >0)
- `requested_at` (timestamp)
- `txn_date` (date)
- `payment_channel` (enum: `HAND_CASH`, `BKASH`, `BANK`, `OTHER`)
- `external_reference` (string, nullable) — trx ID, bank ref, etc.
- `notes` (string, nullable) — e.g. handoff instructions
- `attachment_id` (nullable FK -> FileAttachment)
- `status` (enum: `PENDING`, `APPROVED`, `REJECTED`)
- `reviewed_by` (nullable FK -> User)
- `reviewed_at` (nullable timestamp)
- `rejection_reason` (nullable string)
- `resulting_ledger_id` (nullable FK -> MemberCapitalLedgerEntry)

5.5 Entity: FileAttachment

Required fields:

- `file_id` (uuid, PK)
- `uploaded_by` (FK -> User)
- `mime_type` (string)

- `byte_size` (int)
- `storage_key` (string) — opaque path or object key
- `created_at` (timestamp)

5.6 Entity: MemberCapitalLedgerEntry

Required fields:

- `ledger_id` (uuid, PK)
- `user_id` (FK -> User)
- `entry_type` (enum: SUBMISSION, WITHDRAW, ADJUSTMENT, DISTRIBUTION, DISTRIBUTION_REVERSAL)
- `amount` (decimal(18,2), signed)
- `currency` (default BDT)
- `txn_date` (date)
- `reference_type` (enum: INVESTMENT, MANUAL, SYSTEM, SUBMISSION_REQUEST)
- `reference_id` (nullable string/uuid)
- `submission_request_id` (nullable FK -> CapitalSubmissionRequest) — set when entry originates from FR-04 approval
- `comment` (string, nullable)
- `created_by` (FK -> User)
- `created_at`

5.7 Entity: Investment

Required fields:

- `investment_id` (uuid, PK)
- `title` (string)
- `invested_to` (string)
- `invested_amount` (decimal(18,2), >0)
- `created_date` (date)
- `fund_released_at` (nullable timestamp) — set by Accounts before `OPEN`
- `fund_released_by` (nullable FK -> User)
- `close_date` (nullable date)
- `return_amount` (nullable decimal(18,2))
- `pnl_amount` (nullable decimal(18,2))
- `status` (enum: `DRAFT`, `OPEN`, `CLOSED`, `DISTRIBUTED`, `REVERSED`)
- `comment` (nullable string)
- `created_by`, `created_at`, `updated_at`

Business rule:

- Transition `DRAFT` → `OPEN` requires `fund_released_at` populated (Finance creates in `DRAFT`; Accounts releases funds). Snapshot is captured on transition to `OPEN`.

5.8 Entity: InvestmentSnapshotHeader

Required fields:

- `snapshot_id` (uuid, PK)
- `investment_id` (FK -> Investment, unique)
- `total_capital` (decimal(18,2), >0)
- `member_count` (int, >0)
- `snapshot_time` (timestamp)
- `created_by` (FK -> User)

5.9 Entity: InvestmentSnapshotLine

Required fields:

- `snapshot_line_id` (uuid, PK)
- `snapshot_id` (FK -> InvestmentSnapshotHeader)
- `user_id` (FK -> User)
- `capital_at_snapshot` (decimal(18,2), >=0)
- `ratio` (decimal(18,10), >=0)

Unique:

- (`snapshot_id`, `user_id`)

5.10 Entity: InvestmentDistribution

Required fields:

- `distribution_id` (uuid, PK)
- `investment_id` (FK -> Investment)
- `snapshot_id` (FK -> InvestmentSnapshotHeader)
- `pnl_amount` (decimal(18,2))
- `rounded_total` (decimal(18,2))
- `remainder_applied` (decimal(18,2))
- `status` (enum: POSTED, REVERSED)
- `posted_by` (FK -> User)
- `posted_at`
- `reversed_by` (nullable FK -> User)

- `reversed_at` (nullable timestamp)

5.11 Entity: InvestmentDistributionLine

Required fields:

- `distribution_line_id` (uuid, PK)
- `distribution_id` (FK -> InvestmentDistribution)
- `user_id` (FK -> User)
- `ratio_used` (decimal(18,10))
- `share_amount` (decimal(18,2), signed)
- `ledger_id` (FK -> MemberCapitalLedgerEntry)

Unique:

- (`distribution_id`, `user_id`)

5.12 Entity: AuditLog

Required fields:

- `audit_id` (uuid, PK)
- `entity_name` (string)
- `entity_id` (string/uuid)
- `action` (enum: CREATE, UPDATE, APPROVE, REJECT, RELEASE_FUNDS, CLOSE, DISTRIBUTE, REVERSE, LOGIN, LOGOUT)
- `actor_user_id` (FK -> User)
- `occurred_at` (timestamp)
- `before_json` (json, nullable)
- `after_json` (json, nullable)
- `reason` (nullable string)

6. Non-Functional Requirements

NFR-01 Security

- Passwords shall be hashed with modern one-way algorithm (Argon2 or bcrypt).
- API shall enforce authorization checks server-side.
- Sensitive endpoints shall be rate-limited.
- Personal data shall be protected in transit via TLS.

NFR-02 Reliability and Consistency

- Close-and-distribute operations shall run in database transaction boundaries.

- System shall guarantee atomicity for multi-user ledger postings.
- On failure, transaction rollback shall prevent partial postings.

NFR-03 Performance

- Common list views (members, investments, statements) shall return within 2 seconds for up to 10,000 ledger records.
- Distribution posting for up to 2,000 members shall complete within 10 seconds.

NFR-04 Availability and Backup

- Daily automated database backup with 30-day retention.
- Monthly export archive for reports.

NFR-05 Usability and Localization

- Currency format shall support BDT and Bengali-style display where configured.
- Forms shall validate required fields and numeric ranges.

NFR-06 Auditability

- Every financial mutation shall be reconstructable from immutable entries and audit logs.

NFR-07 File attachments

- Allowed types: at minimum `image/jpeg`, `image/png`, `application/pdf`.
- Maximum upload size: configurable; default **5 MB** per file unless changed at deployment.
- Storage shall be access-controlled: uploader and authorized staff only; members cannot access other users' attachments.
- Optional hardening: virus/malware scanning on upload (deployment-specific).

7. Business Rules

- BR-01: Snapshot is immutable after investment reaches `OPEN` (snapshot captured at that transition).
- BR-02: Distribution must use snapshot ratios, never current live ratios.
- BR-03: One active distribution per investment at a time.
- BR-04: Reversal creates compensating entries only.
- BR-05: Inactive members remain part of historical snapshots and reports.
- BR-06: Pending submission requests do not affect member balance or snapshot capital until approved.
- BR-07: Authorization of a submission request requires a permissioned user; system shall record authorizer and timestamp.
- BR-08: Investment `DRAFT` → `OPEN` requires `fund_released_at` and `fund_released_by` populated.

8. Assumptions and Constraints

- Deployment platform remains open decision.

- System is online-first in MVP.
- Monetary precision fixed at 2 decimal places for posting.
- Administrative actions are trusted but fully auditable.
- Bank/bKash/hand-cash verification is **manual**; trx ID and attachments support evidence only.

9. Acceptance Criteria

- AC-01: Snapshot ratios remain unchanged even after future ledger activity (for a given `OPEN` investment).
- AC-02: Profit/loss distribution equals investment P/L after reconciliation.
- AC-03: Loss distribution produces negative member ledger entries.
- AC-04: Duplicate distribution attempt is blocked.
- AC-05: Reversal nullifies net effect of prior distribution.
- AC-06: Member statement includes transaction source and running balance.
- AC-07: Final member balances can be recalculated from ledger + distributions + reversals.
- AC-08: Pending submission request does not change balance until approval creates a ledger entry.
- AC-09: Approve creates exactly one linked ledger entry; reject creates none.
- AC-10: Super Admin can grant or revoke approval permission; user without permission cannot approve.
- AC-11: Investment snapshot uses authorized capital only; pending requests excluded at snapshot time.

10. Requirement Traceability Matrix (MVP)

Requirement ID	Requirement Summary	Primary Entities	Verification Method
FR-01	Auth, session, permission grants	User, UserPermission, Permission, AuditLog	API test, security test
FR-02	Member lifecycle management	User, AuditLog	UI test, integration test
FR-03	Member submission requests	CapitalSubmissionRequest, FileAttachment	UI + integration test
FR-04	Approve/reject submissions → ledger	CapitalSubmissionRequest, MemberCapitalLedgerEntry, AuditLog	Integration + atomicity test
FR-05	Admin-direct ledger	MemberCapitalLedgerEntry, AuditLog	Integration test
FR-06	Investment DRAFT, fund release, OPEN + snapshot	Investment, InvestmentSnapshotHeader, InvestmentSnapshotLine, AuditLog	Integration test

FR-07	Investment closure	Investment	Integration test
FR-08	Snapshot-based P/L distribution	InvestmentDistribution, InvestmentDistributionLine, MemberCapitalLedgerEntry	Unit + reconciliation test
FR-09	Idempotency and reversal	InvestmentDistribution, MemberCapitalLedgerEntry, AuditLog	Integration + negative test
FR-10	Statements, reports, approval queue	All reporting entities	UI test, report validation
FR-11	Immutable audit trail	AuditLog	Audit query test
NFR-01	Security controls	All auth endpoints	Pen test checklist
NFR-02	Transactional consistency	Investment and ledger posting	Failure simulation test
NFR-03	Performance thresholds	Query/report endpoints	Load test
NFR-04	Backup and retention	Database backup jobs	Ops runbook validation
NFR-07	File attachments	FileAttachment	Upload/security test

11. SDLC Execution Plan

Phase 1: Requirements Finalization

- Review this SRS with stakeholders.
- Freeze calculation rules, rounding policy, and role permissions.

Phase 2: System Design

- Prepare architecture diagram, ERD, API contract, and security model.
- Design indexes and data retention policy.

Phase 3: Implementation (MVP)

- Sprint 1: Auth, Super Admin, permissions, users, submission requests, attachments, approval queue.
- Sprint 2: Admin-direct ledger, investments (`DRAFT` / `OPEN`), fund release, snapshots, closure.
- Sprint 3: Distribution engine, reversal, reports, CSV export.

Phase 4: Testing

- Unit tests for formulas and rounding.
- Integration tests for lifecycle and idempotency.
- UAT with historical sample data from `Root`.

Phase 5: Deployment and Training

- Production deployment with backup job.
- Admin training and onboarding guide.

Phase 6: Maintenance

- Bug fix SLA.
- Monthly audit and data quality checks.

12. Risks and Mitigations

- R1: Incorrect historical distribution due to mutable snapshots.
  - Mitigation: Immutable snapshot tables and DB constraints.
- R2: Rounding mismatch in totals.
  - Mitigation: Deterministic remainder assignment policy.
- R3: Unauthorized financial edits.
  - Mitigation: Strict RBAC and immutable audit logs.
- R4: Human data entry errors.
  - Mitigation: Input validation and reversal workflow.
- R5: Stale or forgotten pending submission requests.
  - Mitigation: Approval queue dashboard for Accounts; optional reminders/notifications in a later version.